CODEFRONTIER HACKATHON 2026 / ROUND 2

HandoverX Project Report

Your team's knowledge. Never lost.

TEAM HACKOVERX	**TEAM LEADER** Abhinav Kumar
INSTITUTION NERIST	**DEPARTMENT** Computer Science and Engineering

Report scope

This report describes the working local Round 2 prototype currently in this repository. It distinguishes implemented behavior from planned product scope. No public deployment, database service, AI API, GitHub connection, or external user account is claimed.

LOCAL DEMO Runs with Node.js; no sign-in or API key.	**HANDOVER PATH** Generate, edit, approve, receive, export.	**VERIFICATION** 4 automated checks passed; browser QA pending.

Project team: HACKOVERX | Team leader: Abhinav Kumar | NERIST

1. Problem and product

Original problem statement

"Build a system that converts scattered project activity into a usable handover when a team member leaves."

Problem understanding

Project information is spread across updates, tasks, decisions, and individual memory. A departing teammate can take practical context with them: what is finished, what is blocked, why a choice was made, and what should happen next. The receiving teammate needs a structured, evidence-based starting point.

Target users

- Outgoing contributors preparing to transfer work.
- Incoming teammates who need to take ownership quickly.
- Project leads who need to review responsibilities, open tasks, and risks.

Proposed solution

HandoverX brings project activities, decision reasoning, task status and ownership, and risk mitigation into one workspace. It assembles those recorded facts into an editable handover for review and approval.

Demo project

CampusConnect is a student collaboration and campus information platform. The seeded workspace includes a four-person team, student dashboard and event-registration work, an agreed pilot scope, open deployment access, and notification and accessibility follow-ups.

2. Architecture and implementation

Architecture

The delivered prototype uses plain HTML, CSS, browser JavaScript, and a small Node.js static server. The browser stores project records in localStorage. The only HTTP endpoint is a health check; record changes are not sent to the server.

Browser application Landing page, workspace views, forms, handover editor
↓ local browser state
localStorage Seeded CampusConnect workspace and generated handovers
↔ static delivery
Node.js server Static files + GET /api/health

Technology stack

- Node.js 18 or later with built-in modules; no package installation required.
- HTML, CSS, and browser JavaScript for the interface.
- Browser localStorage for per-browser records and handovers.
- Node built-in test runner for automated checks.

Architecture choices

The originally proposed React, TypeScript, Vite, Express, Prisma, and PostgreSQL stack was simplified to keep the judge demo install-free and independent of external credentials. This prototype has no shared persistence or production authorization.

Database design

No relational database is implemented. The current browser state contains project, member, activity, decision, task, risk, and handover records. A production version should add relational keys and ownership constraints through PostgreSQL and Prisma migrations.

3. Product modules and workflow

Implemented application modules

- Landing page and Try Live Demo action.
- Dashboard metrics, activity timeline, decisions, pending work, and active risks.
- Activity capture with type, date, responsible member, related task, and evidence URL.
- Decision log with rationale, alternatives, outcome, project area, and evidence.
- Task ownership, priority, status, due date, dependencies, completion notes, search, and status filters.
- Risk tracking with severity, probability, impact, owner, mitigation, and status.
- Team roster with responsibilities, skills, and Active, Leaving, or Inactive status.

Core handover workflow

- Choose the departing teammate and receiving teammate.
- Gather the outgoing member's recorded contributions, decisions, completed and pending tasks, and responsibilities; include project-level risks and recorded links.
- Generate ten sections with a deterministic template. Missing data is marked "Information not recorded."
- Edit sections, save as a draft or under-review handover, and approve.
- Open a receiving-member reading view.
- Download Markdown or print and save to PDF in the browser.

AI implementation

An external LLM is not connected in this build. The fallback is the only generator: it formats current workspace records into structured sections and does not rely on an API key. Four automated checks cover the expected project facts, explicit gaps, member-specific task inclusion, and HTML escaping. No claim of AI model generation is made.

Evidence and screenshots

The report includes no fabricated product screenshots. The browser tool could not connect to the local server, and its URL policy rejected local-file preview. Therefore live desktop and mobile captures remain pending. This is also why visual layout and the full click-through workflow are marked unverified.

4. Security, tests, and delivery

Security considerations

The server adds a Content Security Policy, X-Content-Type-Options: nosniff, and a referrer policy. User-entered text is HTML-escaped before insertion into the interface. The prototype has no accounts, auth tokens, multi-user access, or server-side record writes. It should not be used to store private project data or exposed as a production shared workspace without authentication, authorization, validation, rate limiting, and a persistent database.

Testing performed

On 10 November 2026, `npm run check` parsed both application and server JavaScript successfully. `npm test` passed 4 tests: record-backed generator content, explicit missing-information labeling, departing-member task scoping, and HTML escaping. Browser end-to-end, responsive visual review, print/PDF output, console, and network checks were not completed because the local browser preview was blocked.

Deployment information

No public URL or hosted database has been created. A Render service blueprint is prepared in `render.yaml`, but it has not been deployed. Run locally with `npm start`, then open `http://localhost:4173`. The optional PORT variable changes the default port. No API key is required. Public deployment and repository publication remain pending.

Challenges encountered

The project directory was initially empty. A local syntax issue was found and fixed before tests passed. The provided environment blocked browser access to both localhost and file URLs, preventing screenshot capture and full visual workflow verification.

5. Scope changes and next round

Changes from Round 1 proposal

The supplied product brief was the only proposal source available; no separate Round 1 submission artifact was present. The product goal and CampusConnect example remain aligned with that brief. This implementation simplified the planned stack and hosting to plain browser JavaScript, a local Node server, and localStorage; deterministic templates replace LLM calls. GitHub, PostgreSQL, Prisma, user accounts, public share links, and deployment are not implemented. The changes prioritize a no-setup demo but limit the product to one browser.

Current limitations

- No server-backed persistence, authentication, access control, or multi-user workspace.
- No React/TypeScript, Express API, Prisma schema, or PostgreSQL migrations.
- No Gemini API integration, GitHub connection, or imported evidence.
- No public deployment URL or repository URL.
- Browser end-to-end and responsive visual checks are pending; PDF printing has not been verified.

Three major Round 3 improvements

- Continuous knowledge capture: opt-in read-only connectors with a reviewed import queue, evidence links, webhook verification, and deduplication.
- Intelligent knowledge graph: typed relations among people, decisions, activities, tasks, dependencies, and risks, with traceable sources for handover statements.
- Advanced ownership and risk intelligence: recipient acceptance, reviewer sign-off, transfer deadlines, unresolved-item escalation, and transparent risk trends based on recorded data.

Conclusion

The local Round 2 prototype demonstrates the core handover concept with realistic CampusConnect records and a deterministic report workflow. It runs without sign-in or API keys and has four passing automated checks. The next submission step is to complete browser visual verification, add actual screenshots, and build a secure persistent backend before publishing a public demo.

Prepared for CodeFrontier Hackathon 2026 by HACKOVERX